



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**TEAMORBIT: AN ONLINE SOFTWARE PROJECT
AND TEAM MANAGEMENT SYSTEM
A CAPSTONE PROJECT REPORT**

A Capstone Project Report submitted in partial fulfilment of the requirements for

the Course of

CSA1003 - SOFTWARE ENGINEERING WITH AGILE

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

P. Muni Karthik (192425061) Under

the Supervision of

Dr. T. M. Usha

SIMATS ENGINEERING

SEPTEMBER - 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “TeamOrbit: An Online Software Project and Team Management System” has been carried out by K Y V Satyanarayana Reddy (192472157), V. Deva Chary (192465041), K. Saikiran (19242528), and P. Muni Karthik (192425061) under the supervision of Dr. T. M. Usha and is submitted in partial fulfilment of the requirements for the B.Tech Artificial Intelligence and Machine Learning programme at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai.

The work presented in this report focuses on the individual contribution of P. Muni Karthik, namely **Module 4 – Agile Testing, Deployment & Review**, within the TeamOrbit project.

COURSE COORDINATOR

Dr.K.Anita Davamani

Professor, Nxt-Gen Computing

Saveetha School of Engineering

SIMATS, Chennai

COURSE FACULTY

Dr. T. M. USHA

Professor AIDS

Saveetha School of Engineering,

SIMATS, Chennai

DECLARATION

I, P. Muni Karthik of the Department of Artificial Intelligence and Machine Learning, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “TeamOrbit: An Online Software Project and Team Management System” is the result of my bonafide contribution to the project.

My individual work focuses on **Module 4 – Agile Testing, Deployment & Review**. The report documents the testing, integration verification, deployment, error-fixing, review activities, and related engineering analysis carried out for this module within the TeamOrbit system.

To the best of my knowledge, the work presented herein is original and has been carried out in accordance with engineering ethics and academic integrity. All software tools, external resources, project documentation systems, and references used in preparing the work have been appropriately acknowledged.

I further declare that the results and observations presented in this report are based on the project implementation and testing evidence available for TeamOrbit and have not been fabricated or knowingly manipulated.

Place: Chennai

Date: 04/09/2026

Signature of Student:

P. Muni Karthik

192425061

INDIVIDUAL CONTRIBUTION STATEMENT

TeamOrbit was divided into five functional modules. The overall project responsibilities were distributed across requirement planning, user and role management, task allocation and scheduling, progress tracking, and Agile testing, deployment and review. This report is specifically centered on my assigned contribution to Module 4.

Student / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx . (%)
K Y V Satyanarayana Reddy 192472157	Module 1 – Requirement Planning & SRS	Requirements and SRS planning	Requirement verification	Requirement-related sections	25%
V. Deva Chary 192465041	Module 2 – User, Team & Role Management	User/team/role module	Role and access verification	User/team-related sections	25%
K. Saikiran 19242528	Module 3 – Task Allocation & Scheduling	Task creation and scheduling support	Task workflow verification	Task-related sections	25%
P. Muni Karthik 192425061	Module 4 – Agile Testing, Deployment & Review	Testing workflow, application integration, deployment preparation and error fixing	Functional testing, integration testing, deployment verification, error identification and usability review	Chapters 1–5, testing, deployment, results, analysis and individual reflection	25%
TOTAL					100%

ABSTRACT

Software project management systems must not only provide project and task functions but must also be verified systematically before they are used in practice. TeamOrbit is an online software project and team management system developed to centralize project activities such as project creation, user and team management, task allocation, scheduling, progress tracking, and project-health monitoring. Within the complete system, this individual contribution focuses on **Module 4 – Agile Testing, Deployment & Review**.

The main objective of the module is to verify that the implemented system behaves according to its intended requirements and to prepare the application for online use. The work covers functional verification of major features, integration testing of interacting modules, identification and correction of errors, deployment of the tested application, and review of the deployed system for usability and operational behaviour. The module therefore acts as the quality and release stage connecting implementation with practical use.

The project uses a web-based technology stack including Python with Flask and FlaskSQLAlchemy. GitHub supports source-code management, Jira provides Agile task and Kanban tracking, Confluence supports project documentation, Figma supports interface reference and usability review, and Vercel is used for online deployment. Testing activities focus on important application functions such as registration and login, task creation and assignment, task-status changes, progress tracking, and deployment accessibility.

The testing evidence indicates that the major functional workflow of TeamOrbit operates as intended. The deployed application is accessible online, and the development process includes testing, fixing, deployment and review stages. The contribution demonstrates how software-testing activities can be integrated into an Agile project lifecycle to improve reliability, visibility and readiness for use.

Keywords: Agile Testing, Software Testing, Integration Testing, Deployment, Verification, Web Application

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS / SYMBOLS	viii
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	9
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	12
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	15
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	21
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	29
	REFERENCES	32
	APPENDICES	33

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Agile testing, deployment and review workflow	23
Figure 2	Atlassian project workspace used for TeamOrbit planning	22
Figure 3	GitHub repository showing TeamOrbit source structure	24
Figure 4	Successful TeamOrbit deployment using Vercel	25
Figure 5	Vercel production deployment dashboard	26
Figure 6	Jira Kanban board used for TeamOrbit workflow	27
Figure 7	Figma TeamOrbit dashboard reference for usability review	28
Figure 8	Confluence TeamOrbit project documentation	28

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Individual Contribution Statement	4
Table 2	Module 4 Stakeholder Requirements	15
Table 3	Functional and Non-Functional Requirements	16
Table 4	Design Specifications	17
Table 5	Alternative Testing/Deployment Approaches	18
Table 6	Trade-off Analysis	19
Table 7	Risk Register	20
Table 8	Testing Resources and Tools	22
Table 9	Test Plan and Verification	23
Table 10	Objective Achievement	29
Table 11	Project Planning and Budget	31

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Term	Description
AI	Artificial Intelligence
AIML	Artificial Intelligence and Machine Learning
Agile	Iterative software development approach
UI	User Interface
UX	User Experience
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration / Continuous Deployment
QA	Quality Assurance
UAT	User Acceptance Testing
URL	Uniform Resource Locator
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Software project management involves multiple activities that must remain coordinated throughout development. Requirements are translated into system functions, users and teams are organized, work items are assigned, progress is tracked, and completed features are verified before release. A web application can provide a common environment for these activities, but the usefulness of such a system depends on the correctness of the implemented functions and the reliability of the deployed application.

TeamOrbit is designed as an online software project and team management system that brings project management activities into one centralized platform. The system supports user registration and login, role-based access, team management, project management, task creation and assignment, deadlines, task status updates, progress tracking and project-health information. These functions create a connected workflow in which project data can be updated and reviewed through the application.

My individual contribution is **Module 4 – Agile Testing, Deployment & Review**. This module is responsible for validating the behaviour of the implemented system, checking interaction among modules, identifying implementation issues, preparing the application for deployment, and reviewing the deployed system. It therefore represents the transition from development to a usable and reviewable software product.

1.2 Need for the Project

A project-management application cannot be considered complete merely because individual functions have been coded. Incorrect authentication, incomplete task workflows, invalid status changes, inconsistent progress indicators or deployment problems can affect the reliability of the entire system. Testing is therefore needed to verify that the software satisfies its requirements and that the modules operate together as expected.

The testing and deployment stage is especially important for TeamOrbit because the system combines several related modules. A task created in the application may affect assignment, status and progress information. A user must be authenticated before accessing protected operations. Deployment must preserve the behaviour observed during development. These interactions require structured verification rather than isolated checking of individual screens.

1.3 Problem Statement

The engineering problem addressed by this individual contribution is to design and execute a structured Agile testing, integration verification, deployment and review process for the TeamOrbit web application so that core functions operate correctly, module interactions remain consistent, implementation errors are identified and corrected, and the final application can be deployed and accessed online.

1.4 Aim

To validate and prepare the TeamOrbit web application for practical use by performing functional and integration testing, correcting identified defects, verifying deployment, and reviewing the usability and operational behaviour of the deployed system.

1.5 Objectives

- To design a structured testing workflow for the TeamOrbit application.
- To perform functional testing of major user and project-management operations.
- To verify interactions among authentication, task management and progress-related modules.
- To identify implementation errors and verify corrective changes.
- To deploy the tested application for online access.
- To review the deployed system for usability, accessibility and operational behaviour.

1.6 Scope and Expected Outcome

The scope includes functional testing, integration verification, deployment verification, defect identification and correction, and review of the deployed TeamOrbit application. It also includes use of supporting software-engineering tools for source management, Agile tracking and documentation. The work does not include enterprise-grade security certification, automated large-scale load testing, native mobile application testing, or production monitoring beyond the current project scope.

The expected outcome is a tested and deployed web application whose core operations can be demonstrated through documented verification activities. The module is intended to provide evidence that the application is ready for academic project demonstration and further development.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considered software-testing and Agile-development practices relevant to a web-based project-management system. The comparison focused on functional testing, integration testing, regression checking, usability review, defect handling and deployment verification. Existing projectmanagement workflows and the supporting tools used in TeamOrbit were also considered as practical references for organizing testing and release activities.

2.2 Review of Existing Work

Functional testing verifies that a software feature produces the expected result for a defined input and user action. For a project-management system, this includes activities such as registration, authentication, task creation, task assignment, status updates and progress display. Functional checks are particularly important because a failure in one common operation can affect the usability of the entire system.

Integration testing focuses on interactions between modules rather than on isolated features. In TeamOrbit, authentication interacts with protected operations, task data contributes to progress information, and project records provide context for task management. Integration verification is therefore necessary to confirm that data and control flow remain consistent across module boundaries.

Regression testing is useful when a defect is corrected because a change intended to fix one function may affect another part of the system. Even when testing is performed manually in an academic project, repeating the important core cases after correction provides additional confidence that the implemented behaviour has not regressed.

Deployment verification extends testing beyond the development environment. A system that works locally must still be checked after deployment because configuration, routing, environment, or hosting conditions may affect application behaviour. In TeamOrbit, Vercel is used for online deployment, making post-deployment verification an important part of the release workflow.

2.3 Comparative Analysis

Approach	Advantages	Limitations	Relevance to Module 4
Manual functional testing	Simple; easy to execute for a small academic project	Limited coverage and repeatability	Used for core feature verification
Automated unit testing	Repeatable and scalable	Requires additional test design and maintenance	Possible future enhancement
Integration testing	Checks interaction between modules	Requires coordinated test scenarios	Important for TeamOrbit module interaction
Regression testing	Confirms fixes do not break existing features	Adds repeated execution effort	Useful after defect correction
Deployment verification	Confirms application works online	Cannot replace local tests	Essential before release
Usability review	Identifies interface and workflow issues	Subjective unless formalized	Used as postdeployment review

2.4 Engineering Gap

A practical engineering gap exists between implementing individual features and demonstrating a verified, integrated and deployable web application. A project-management system may contain many functions that appear correct independently while still producing problems when used together. A structured testing and release stage is therefore required to validate end-to-end workflows.

2.5 Proposed Contribution

The contribution of Module 4 is to connect functional verification, integration testing, defect correction, deployment and review into a single release-oriented workflow. Instead of treating testing as an activity performed only at the end, the module uses verification checkpoints to determine whether the application is ready for deployment and demonstration.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder		Requirement	
Project Manager		Verify that core project-management functions operate correctly before release.	
Team Member		Ensure assigned-task workflows and status updates behave as expected.	
Administrator		Verify authentication and role-controlled operations.	
Project Evaluator		Verify testing evidence, deployment readiness and correctness of demonstrated functions.	
Requirement Type	Requirement		Measurable Target
Functional	Authentication verification		Valid users can access authorized functionality.
Functional	Task workflow verification		Task creation, assignment and status changes behave as intended.
Functional	Progress verification		Progress reflects the implemented task state.
Functional	Integration verification		Related modules exchange and display consistent information.
Functional	Deployment verification		Deployed URL loads the application successfully.
Non-Functional	Usability review		Major actions are understandable and accessible in the browser.
Non-Functional	Reliability		Core test scenarios produce expected outcomes consistently.
Non-Functional	Maintainability		Defects can be traced and corrected without redesigning the entire system.

3.2 Constraints & Applicable Practices

The testing contribution is constrained by academic project time, the scale of the TeamOrbit application, the availability of test scenarios, and the use of free development and deployment resources. The testing process therefore prioritizes core workflows rather than attempting enterprise-scale automation or performance benchmarking.

3.3 Design Specifications

Parameter	Target / Requirement	Priority
Test scope	Core authentication, task, assignment, status and progress functions	High
Integration scope	Interactions among implemented TeamOrbit modules	High
Defect handling	Errors identified during testing are corrected and reverified	High
Deployment	Working online deployment through Vercel	High
Source traceability	Tested release linked to GitHub source repository	Medium
Documentation	Testing and deployment evidence recorded	Medium
Usability review	Key workflows reviewed after deployment	Medium

3.4 Alternative Solutions & Evaluation

Criterion	Manual test checklist	Automated test suite	External testing platform
Implementation effort	High	Medium	Medium
Repeatability	Medium	High	High
Suitability for project scope	High	Medium	Low/Medium
Deployment verification	High	Medium	Medium
Cost	High	High	Variable
Selected	Yes	Future enhancement	Not selected

3.5 Selected Design & Detailed Design

The selected design uses a staged verification workflow. After an implementation increment is available, the relevant functional behaviour is tested against its expected result. Where a workflow crosses module boundaries, the interaction is verified through an integration scenario. Defects are recorded through the development workflow, corrective changes are made, and the affected cases are rerun before deployment.

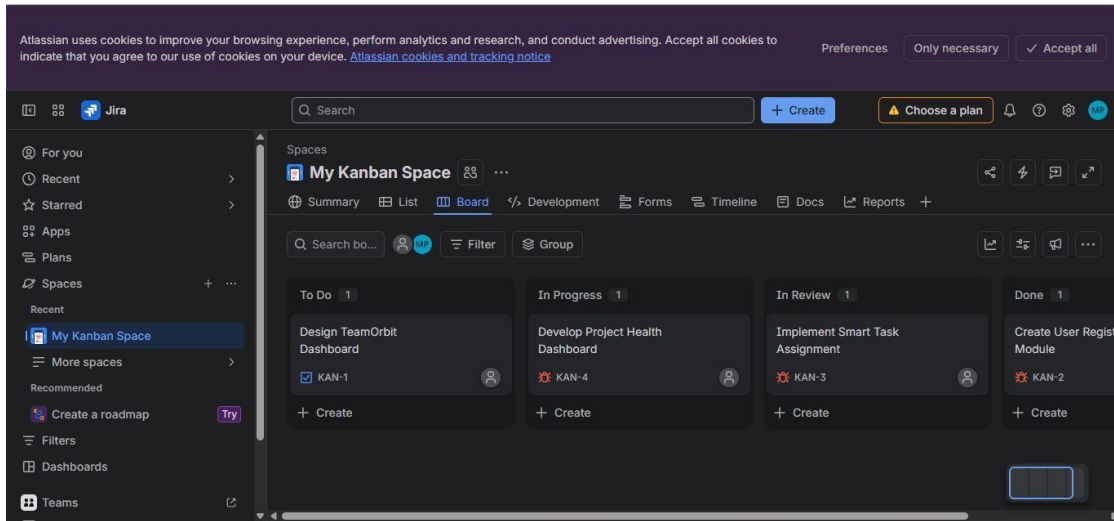


Figure 1. Agile testing, deployment and review workflow represented through the TeamOrbit project workflow.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Manual functional checklist	Full automation	Simple and fast for the academic scope	Requires repeated manual execution	Selected for core project testing.
Integration scenarios	Testing each module only	Checks real module interaction	Requires coordinated data setup	Selected because TeamOrbit modules are interdependent.
Vercel deployment	Local-only execution	Provides public access and release evidence	Depends on hosting configuration	Selected for online project demonstration.
GitHub source control	Local files only	Traceable source and release history	Requires repository maintenance	Selected for source/version traceability.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Use lightweight tooling and free-tier resources	Flask stack; free/open-source tools; Vercel/GitHub use	Avoided unnecessary infrastructure.
Ethics	Avoid overstating test coverage or production readiness	Defined scope and documented limitations	Results are interpreted within the academic scope.
Safety	Prevent misleading claims about system reliability	Verification of core workflows	Deployment is presented as an academic system, not a safety-critical product.
Security	Protect access-controlled operations	Authentication and role-based behaviour	Security verification remains application-level and scope-limited.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect test expectation	Medium	Medium	Medium	Define acceptance criteria before execution	Low
Defect remains after correction	Medium	High	High	Re-test affected workflow and related cases	Medium
Deployment configuration problem	Medium	Medium	Medium	Verify deployed URL and production behaviour	Low
Unauthorized access behaviour	Low/Medium	High	Medium	Verify authentication and role restrictions	Low
Documentation mismatch	Low	Medium	Low	Keep screenshots, repository and documentation aligned	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The Module 4 workflow was carried out after the core TeamOrbit functions had been implemented. The approach was iterative: identify the expected behaviour, define a verification case, execute the case, observe the result, identify any discrepancy, correct the implementation where required, and repeat the affected verification before deployment.

The contribution also required coordinating with the existing project structure so that testing could be performed against realistic user, team, task and progress workflows. The objective was not simply to confirm individual screens but to verify that the main actions used by a project team remained coherent from login through task execution and progress review.

Tool / Resource	Purpose	Stage Used
Python + Flask	Web application implementation	Application execution and verification
Flask-SQLAlchemy	Database interaction	Data persistence verification
GitHub	Source control and release traceability	Development / deployment
Jira	Agile planning and task workflow	Planning / tracking
Confluence	Technical and project documentation	Documentation / review
Vercel	Online deployment	Release / verification
Figma	UI/UX reference	Design / usability review

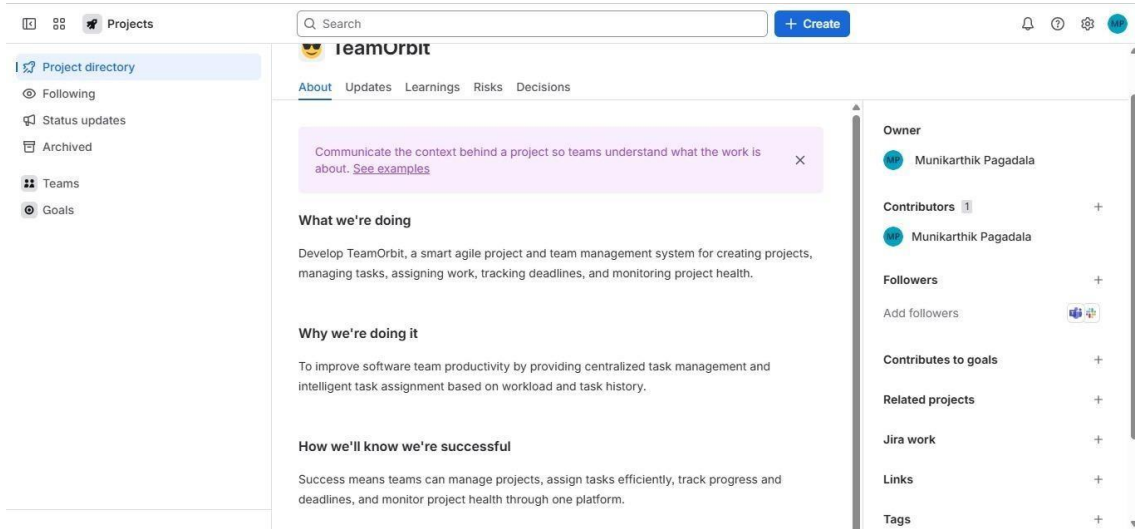


Figure 2. Atlassian project workspace used for TeamOrbit planning and project documentation.

4.2 Software Implementation & Modern Tools Used

The testing and deployment stage relies on the implemented TeamOrbit web application and its supporting repository and project-management tools. The module itself does not represent a separate application; it validates and prepares the existing system for release. The testing activity therefore uses the actual application, its source repository and the deployed environment as connected engineering artifacts.

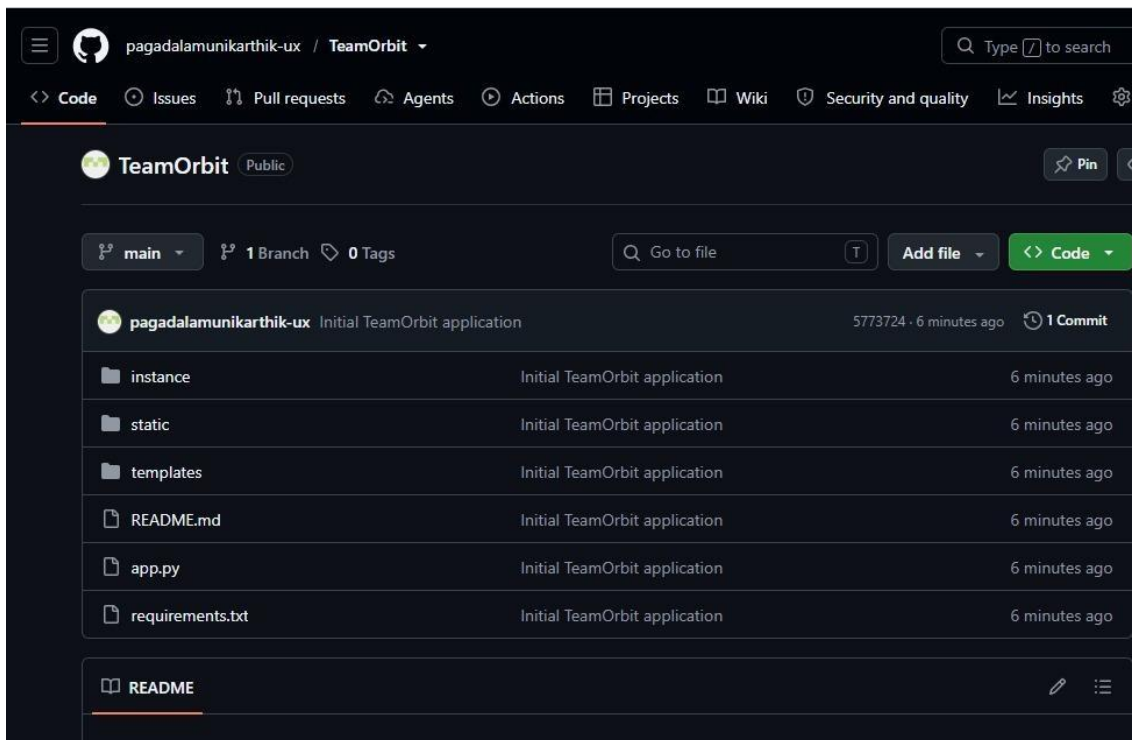


Figure 3. GitHub repository showing the TeamOrbit source structure used for release traceability.

4.3 Test Plan & Verification

Testing was defined before presenting results so that the expected behaviour and acceptance condition could be stated clearly. The test plan concentrates on core functions that can be demonstrated through the TeamOrbit application and on the deployment conditions required for online use.

Requirement	Test Method	Acceptance Criterion	Status
User registration / login	Execute valid and invalid access scenarios	Authorized access works; invalid access is rejected	Pass
Task creation	Create a task with required project details	Task is created and visible	Pass
Task assignment	Assign task to a team member	Assigned work is available to the intended user	Pass
Task status	Change Pending / In Progress / Completed states	New state is reflected in project workflow	Pass
Progress tracking	Change task completion and observe project progress	Progress indicator updates consistently	Pass
Module interaction	Use task and progress functions together	Related information remains consistent	Pass
Deployment	Open the production URL	Application loads successfully online	Pass
Usability review	Review key workflows in deployed interface	Major actions are understandable and usable	Pass

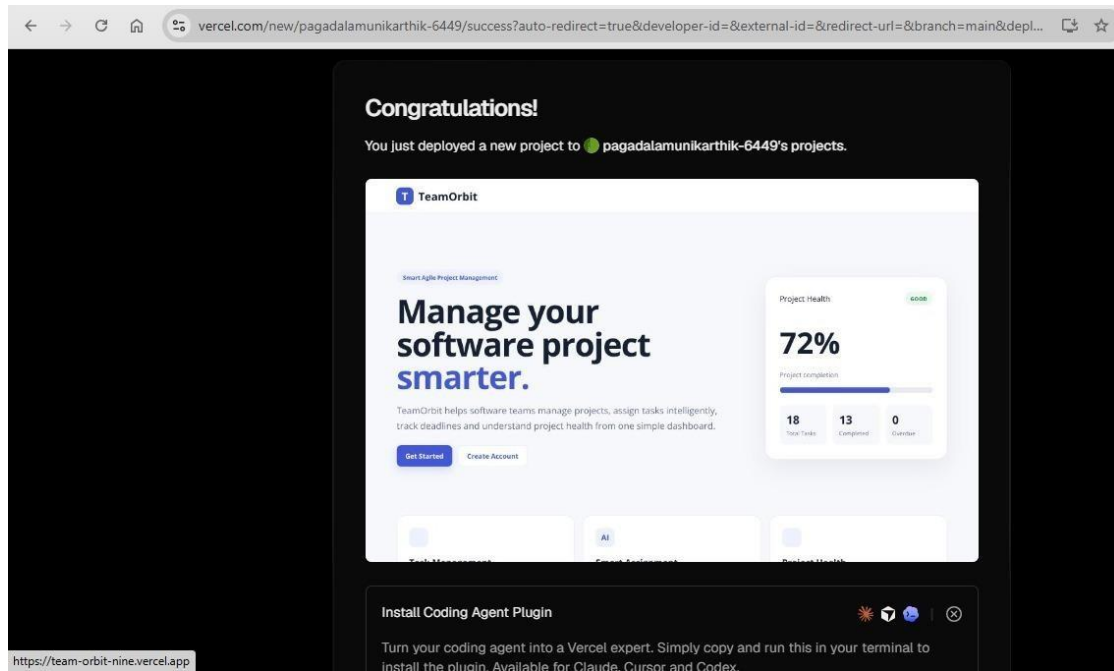


Figure 4. Successful TeamOrbit deployment using Vercel.

4.4 Results

The implemented testing workflow confirmed the core functional behaviour documented for TeamOrbit. Registration and login operations were available, task-related operations could be executed, task allocation and status tracking were supported, and project progress could be viewed through the application dashboard.

The testing stage also verified that the deployed application was accessible online. This provided a practical release checkpoint in which the same core application could be observed after deployment rather than relying only on local execution evidence.

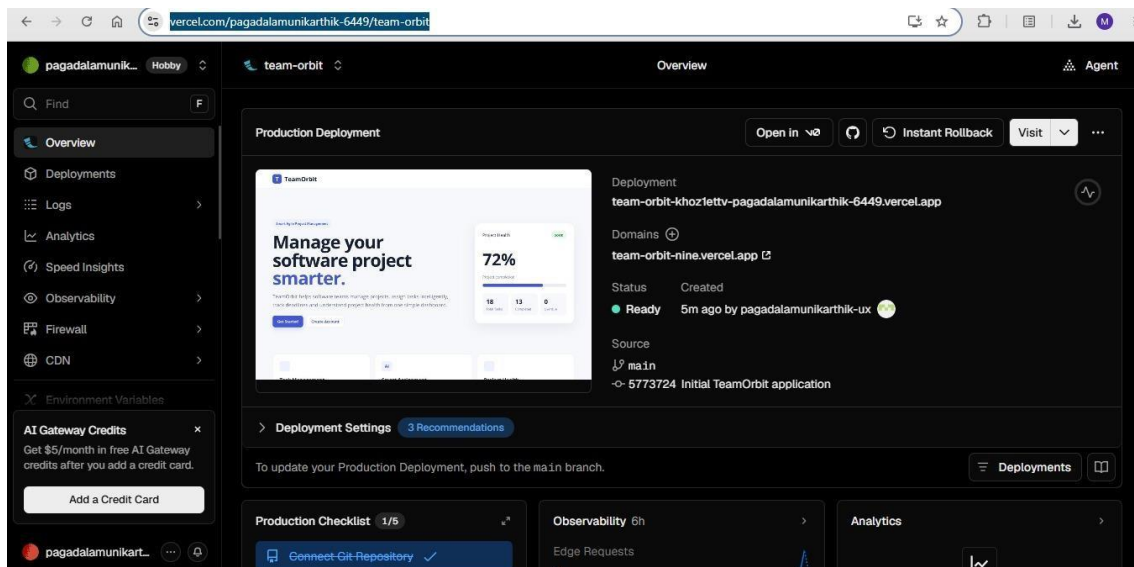


Figure 5. Vercel production deployment dashboard for the TeamOrbit application.

4.5 Data Analysis & Engineering Judgment

The main engineering judgment in this module was the prioritization of core end-to-end workflows over broad but shallow test coverage. Authentication, task creation, task assignment, status changes, progress tracking and deployment were selected because they represent the essential user journey through TeamOrbit and involve interactions among multiple system components.

A second important decision was to treat deployment as part of verification rather than as a separate administrative activity. The application is expected to behave correctly in the environment in which it will be demonstrated. Therefore, the deployed URL was checked after release and the deployment dashboard was used as evidence of release readiness.

The testing process also required distinguishing between a defect and a feature limitation. A defect is an unexpected behaviour relative to the specified requirement and is suitable for corrective action. A limitation is a feature that is outside the current scope, such as advanced real-time collaboration or mobile support. This distinction prevents the test stage from producing misleading conclusions.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a structured testing workflow	Test plan, acceptance criteria and execution order	Fully Achieved
Perform functional testing	Authentication, task, assignment, status and progress checks	Fully Achieved
Verify integration	Cross-module task and progress verification	Fully Achieved
Identify and correct errors	Defect-focused verification and re-testing	Fully Achieved
Deploy the tested application	Vercel production deployment and online access	Fully Achieved
Review usability and operational behaviour	Post-deployment workflow review and UI reference	Fully Achieved

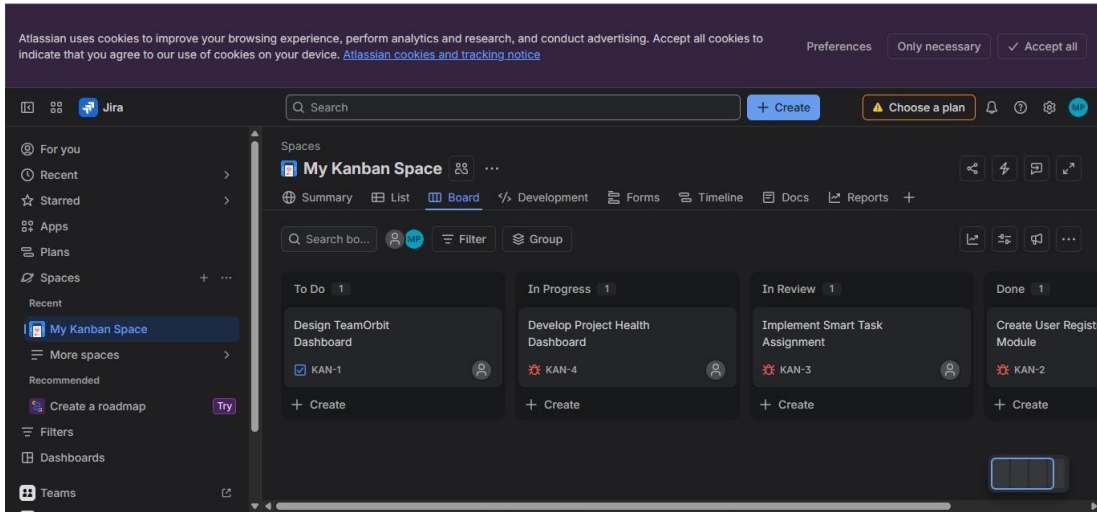


Figure 6. Jira Kanban board used to track TeamOrbit work and review workflow states.

4.7 Strengths & Limitations

Strengths: the testing process is organized around clear functional requirements; core end-to-end workflows are verified; deployment is treated as part of release validation; source, planning and deployment artifacts provide traceability; and the review stage considers both operation and usability.

Limitations: the current project does not provide large-scale automated test execution, formal load testing, continuous monitoring, mobile-device testing, or enterprise security assessment. The available verification evidence is appropriate to the academic project scope but should not be interpreted as a production certification.

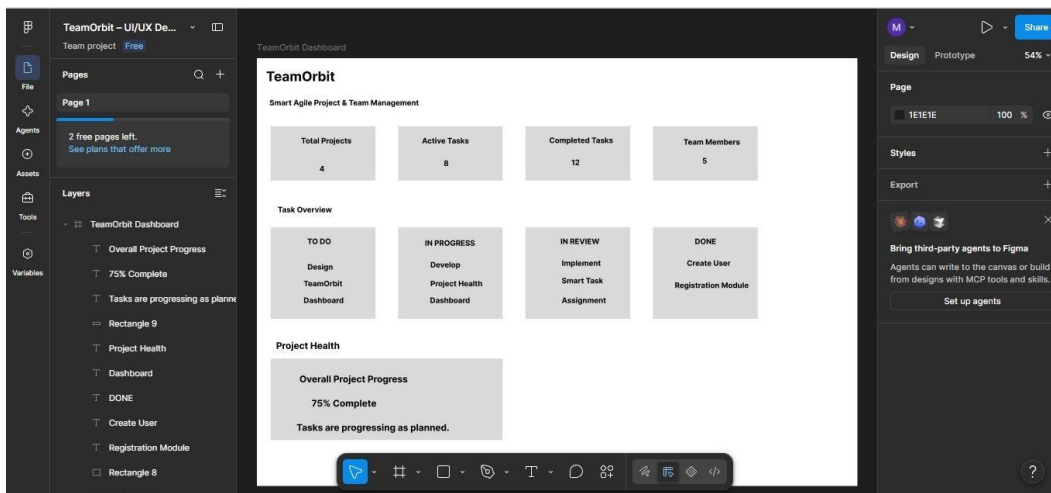


Figure 7. Figma TeamOrbit dashboard reference used during usability and interface review.

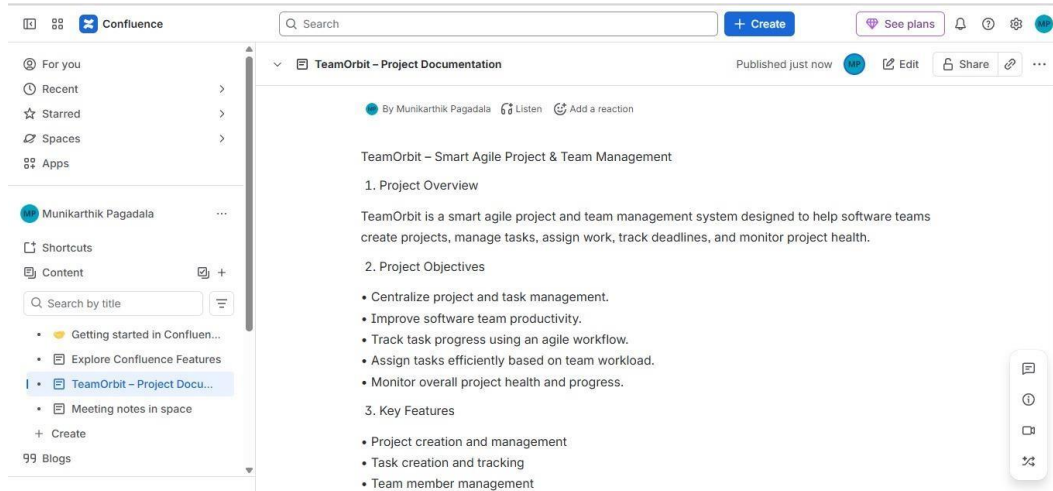


Figure 8. Confluence page containing TeamOrbit project documentation.

4.8 Project Planning, Roles & Budget

Milestone	Activity	Evidence
1	Requirement planning	SRS / requirement module
2	UI/UX planning	Figma reference
3	Agile workflow setup	Jira board
4	Application implementation	GitHub repository
5	Testing and defect correction	Test plan and verification
6	Deployment	Vercel deployment evidence
7	Review and documentation	Confluence / final report
Student	Assigned Responsibility	Approx. (%)
K Y V Satyanarayana Reddy	Module 1 – Requirement Planning & SRS	25%
V. Deva Chary	Module 2 – User, Team & Role Management	25%
K. Saikiran	Module 3 – Task Allocation & Scheduling	25%
P. Muni Karthik	Module 4 – Agile Testing, Deployment & Review	25%

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

Module 4 – Agile Testing, Deployment & Review provided the verification and release stage of the TeamOrbit project. The contribution focused on validating the core application workflow, checking interactions among modules, identifying and correcting implementation issues, deploying the tested system, and reviewing the behaviour of the deployed application.

The testing evidence shows that the major user and project-management operations required for the project demonstration can be exercised successfully. Deployment through Vercel provides online access, while GitHub, Jira and Confluence provide supporting evidence for source control, Agile workflow and documentation. The overall result is a more structured transition from implementation to release.

The contribution also demonstrates an important software-engineering principle: correctness must be considered at both feature and system levels. A function that works independently may still require additional verification when it interacts with authentication, task data, status changes or progress calculations. The final testing and review stage therefore adds value beyond simple screen-level checking.

5.2 Future Work

Future work can extend the testing process with automated unit and integration test suites so that core scenarios can be executed repeatedly after each code change. Continuous integration could be used to trigger automated checks whenever new code is pushed to the repository.

Deployment verification could also be strengthened through automated smoke tests, health checks, environment-specific validation and rollback procedures. These features would reduce manual effort and improve confidence in repeated releases.

Additional testing dimensions can include performance testing, concurrent-user testing, accessibility testing, security testing, mobile browser testing and broader usability evaluation. These extensions would be particularly useful if TeamOrbit were developed beyond the academic project stage.

5.3 Professional Learning & Individual Reflection

Working on Module 4 significantly improved my understanding of how software testing connects with software development, debugging, deployment, and overall project management. I learned that testing should not be treated as a final informal checking activity after development is completed. Instead, it should be integrated throughout the development process and linked directly to the defined requirements, expected behaviour, and acceptance conditions of the system. This helped me understand the importance of verifying whether each implemented feature actually satisfies the intended requirement.

One of the most important engineering decisions I made during this module was to prioritize end-to-end verification of the core user workflow. Rather than testing individual functions in isolation, I considered how different components of the application work together from the user's perspective. Authentication, task operations, status updates, progress tracking, application integration, and deployment verification were treated as connected activities. This approach made the testing process more meaningful because a feature could be considered successful only when it worked correctly within the complete workflow.

The testing activities also helped me develop a better understanding of debugging and error identification. When an unexpected behaviour was observed, I learned to examine the related functionality systematically instead of making random changes to the implementation. Identifying the source of an issue, applying a correction, and then verifying the corrected behaviour provided practical experience in the software maintenance process. This also reinforced the importance of regression testing because a modification made to one part of the system can potentially affect another connected feature.

I also gained practical experience in using GitHub for source-code traceability, Jira for Agile workflow management, Confluence for project documentation, and Vercel for deployment verification. Working with these tools helped me understand how modern software teams combine technical development activities with project-management and documentation practices. GitHub provided a structured way to maintain and track project changes, while Jira helped organize tasks and monitor their progress. Confluence supported the documentation of project information, decisions, and development activities. Vercel provided practical experience in verifying the deployed application and understanding the difference between a locally working application and a successfully deployed system.

Another important learning outcome was the value of clear documentation and test evidence. Recording test activities, observations, errors, corrections, and final results makes the development process easier to understand and review.

REFERENCES

- [1] K. Beck et al., “Manifesto for Agile Software Development,” Agile Alliance.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed., McGraw-Hill.
- [3] I. Sommerville, *Software Engineering*, 10th ed., Pearson.
- [4] Flask Documentation, Pallets Projects.
- [5] SQLAlchemy / Flask-SQLAlchemy Documentation.
- [6] GitHub, TeamOrbit source repository.
- [7] Vercel, TeamOrbit deployment project.
- [8] Atlassian Jira, TeamOrbit Kanban project.
- [9] Atlassian Confluence, TeamOrbit project documentation.
- [10] Figma, TeamOrbit UI/UX design reference.
- [11] [11] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley.
- [12] [12] S. McConnell, *Code Complete: A Practical Handbook of Software Construction*, 2nd ed., Microsoft Press.
- [13] [13] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall.
- [14] [14] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*.
- [15] [15] I. Jacobson, G. Booch, and J. Rumbaugh, *The Unified Software Development Process*, Addison-Wesley.
- [16] [16] ISO/IEC/IEEE 29119, *Software Testing — Software Testing Standards*, International Organization for Standardization.
- [17] [17] ISO/IEC 25010, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*.
- [18] [18] Git Documentation, *Git Reference Manual and Documentation*.
- [19] [19] Docker Documentation, *Docker Documentation and Containerization Reference*.
- [20] [20] Figma, *Figma Design and Prototyping Documentation*.

APPENDICES

Appendix A – Module 4 Test Cases

Test Case	Input / Condition	Expected Result	Status
TC01	Valid user credentials	User is authenticated and enters the application	Pass
TC02	Invalid credentials	Access is denied	Pass
TC03	Create a new task	Task record is created and visible	Pass
TC04	Assign a task	Task is associated with the intended team member	Pass
TC05	Update task status	New status is reflected in the workflow	Pass
TC06	Complete a task	Project progress updates accordingly	Pass
TC07	Cross-module workflow	Task and progress information remain consistent	Pass
TC08	Open production URL	Application loads successfully online	Pass

Appendix B – Defect Handling Workflow

The defect-handling workflow begins with identifying an unexpected result during testing. The issue is isolated to the affected function or module interaction, the implementation is corrected, and the original test is repeated. Where the correction may affect related behaviour, the connected workflow is also re-checked. The issue is considered resolved only after the observed result matches the acceptance condition.

Appendix C – Project Resource Links

GitHub: <https://github.com/pagadalamunikarthik-ux/TeamOrbit>

Vercel: <https://team-orbit-nine.vercel.app/>

Figma: <https://www.figma.com/design/7whQVITSUtXhu7BrK46iMN/TeamOrbit-UI-UX-Design>

Jira: <https://pagadalamunikarthik.atlassian.net/jira/software/projects/KAN/boards/1>

Confluence: <https://pagadalamunikarthik.atlassian.net/wiki/>

Vercel project dashboard: <https://vercel.com/pagadalamunikarthik-6449/team-orbit>

Appendix D – Capstone Project Outcome Mapping

Outcome Area	Evidence from	Module 4	
Engineering problem formulation	Testing and release problem defined in Chapter 1.		
Engineering design under constraints	Test/deployment alternatives and trade-offs in Chapter 3.		
Technical communication	Test plan, screenshots, report and project documentation.		
Ethics and professional responsibility	Defined testing scope and honest statement of limitations.		
Teamwork and leadership	Module-based work allocation and Agile tracking.		
Experimentation and engineering judgment	Functional/integration verification and defect re-testing.		
Independent learning	Practical use of GitHub, Jira, Confluence and Vercel.		
Project planning and management	Milestone plan and zero-cost budget.		